including those that are not associated with a terminal session. This option enables monitoring of all the processes running on a system.

```
$ps aux
```

The '-ef' option provides more detailed information compared to the 'aux' option, which gives a more user-friendly output. You can customise the output of the 'ps' command by using various options. Table 1 lists some of the most useful options for the 'ps' command.

Table 1: The 'ps' command and its various options

Command options	Effect
ps -e	Lists all running processes in the system.
ps -f	Displays a detailed list of all running processes started by the user, including the parent process ID, session ID, and the process group ID.
ps -ef	Displays a full list of all running processes with additional details such as the user name, start time, and the command that launched the process.
ps aux	Displays a detailed list of all processes running on the system, including those owned by other users. The output includes the process ID (PID), CPU usage, memory usage, and other relevant details.
ps -C <command>	Displays all processes that are running a particular command.
ps -u <username>	Displays all processes owned by a particular user.
ps -o <columns>	Allows you to specify the columns you want to display in the output.
ps -p <pid>	Displays detailed information about a specific process identified by its process ID.

Killing processes

You can terminate a process that is running on your system by using the 'kill' command along with the PID of the process in Linux.

```
$kill 102353
```

The 'kill' command sends a signal called SIGTERM to the process, asking it to terminate gracefully. If the process does not respond to the SIGTERM signal, the '-9' option can be used with the 'kill' command to send a signal called 'SIGKILL', which forcefully ends the process.

```
$kill -9 102353
```

Killing a process using the 'kill' command could have unintended consequences, such as data loss or other issues if the process is performing a critical task. Therefore, it is important to use the `kill` command with caution and only when necessary to gracefully terminate a process or force it to end immediately. The `kill` command is a powerful tool for stopping processes in Linux, and by using the options, you can manage the system effectively.

'killall' command

The 'killall' command allows you to kill all processes that match a name. For example, if you want to kill all Chrome processes, you can run the following command at the prompt:

```
$killall chrome.
```

The '-u' option is available to kill all processes owned by a particular user. For example, you can end all processes owned by a user named 'ravi' with the following command:

```
$killall -u ravi
```

Parent and child processes

In Linux, each process has a parent process. Upon starting, Linux assigns a process a unique process ID (PID) and a parent process ID (PPID). The PPID is the PID of the process that started the current process. The new process becomes a child process of the process that created it, and inherits many of its properties from the parent process, such as its environment variables and file descriptors.

The benefit of parent and child processes is that they can communicate with each other using inter-process communication (IPC) mechanisms such as pipes, sockets, and shared memory that allow them to work together to perform complex tasks and also facilitate process management. For example, a parent process can signal the child process to terminate if it has stopped responding to it. This prevents the child from consuming too many system resources and causing other issues.

Parent and child processes are a fundamental concept in Linux process management. By understanding how parent and child processes work, you can perform complex tasks and manage system resources effectively.

Keeping background processes around

When you start a process from the command line, it